

The 12% Adoption Problem: AI PR-Review Agent Product Case

Practice date: 2026-10-09. Problem-solving type: product case (EM-flavored product sense). Worked interactively, interviewer-style. Score: 8/10.

The scenario

You are the EM of the team that owns the AI agent that auto-drafts pull-request reviews inside the company's monorepo. Adoption is stuck at 12% of engineers after six months. Draft quality is good. Latency is fine.

Three questions: (1) what do you measure beyond adoption to find whether this is awareness, workflow, or trust; (2) pick one hypothesis to bet the quarter on and scope the MVP; (3) what do you deliberately not build.

The worked solution

0. Frame it before touching it. "Quality is good and latency is fine, so this is not a model problem - it's a distribution problem. My job is to find where in the distribution the drop happens." Say this first. It tells the interviewer you won't spend the quarter tuning a model that isn't broken.

1. Is the 12% real? (before anything else) Check how the metric is produced before trusting it. What counts as "adoption" - engineers who ever triggered it once, weekly actives, share of PRs with a draft viewed? A bad denominator poisons every conclusion downstream. Senior instinct: verify instrumentation before forming hypotheses.

2. Cohort the 12% - the single highest-value cut. Is it the same 12% every week (sticky core ? acquisition problem: everyone else never starts) or a rotating 12% (people try it and drop ? retention/value problem)? This one segmentation changes the entire plan, and most candidates never ask the question.

3. Funnel the pipeline before interviewing anyone. Think of it as a pipeline: PR opened ? draft generated ? draft viewed ? draft accepted / edited / dismissed ? PR merged. Measure conversion at each stage. The stage with the cliff names the problem class for free:

- Nobody generates drafts ? awareness.
- Drafts generated but never viewed ? workflow (it's not where the developer already works).
- Viewed but dismissed or heavily edited ? trust (or perceived quality).

4. Read trust off behavior, not just interviews. Trust is hard to ask about directly - nobody says "I don't trust it," they just stop clicking. Behavioral proxies: edit distance on drafts before merge (heavy edits = low trust), dismiss rate, and whether a full human re-review still happens after the AI review. If the AI review doesn't reduce human review effort, it's decoration.

5. Unaided awareness check. A short survey that never names the tool: "how do you get your PRs reviewed today?" If engineers describe the old workflow with no mention of the agent, it's awareness. Priming them with the tool's name measures politeness, not awareness.

6. Stratified interviews last, not first. Only after the data: 5 power users, 5 tried-and-dropped, 5 never-tried - each group gets different questions. Random sampling mixes the signals; stratification separates them.

The bet

Workflow. Rank the hypotheses: (a) awareness - possible, but six months in with presumably

some launch comms, check it with the survey rather than assume it; (b) workflow - the draft isn't where the developer already works: buried behind a button, or arriving after human reviewers already weighed in; (c) trust - weakened by the stated facts (quality confirmed good, latency fine). Bet on (b), verify (a) cheaply, keep (c) monitored via the behavioral proxies.

The MVP

Change the mechanism, not the model. **Auto-post the draft as a PR comment the moment the PR opens** (or as inline annotations), so it lands before human reviewers arrive and exactly where the developer already reads. Scope it to stable repos covering roughly 20% of devs - stable means low variance, so the read on the metric is clean.

- In: auto-posting, timing (before human review), the 20% pilot population.
- Out: new review capabilities, new languages, configurability, per-team customization.
- Success in six weeks: human acceptance rate of drafts, and - the metric that matters - whether human review cycles per PR drop. If the AI review doesn't replace human effort, it isn't helping.

What you deliberately don't build

New features - more languages, deeper analysis, custom rules - until the data says features are the blocker. And no big internal marketing campaign before knowing whether the problem is awareness: if the funnel says workflow, marketing is wasted spend. Defend it in product review with the funnel: "the data says the drop is here, so that's where the quarter goes."

Scorecard

What went well (8/10): diagnosed before prescribing; the cohort question on the 12% (same users vs. churn) was the best instinct in the answer; instrument verification before hypotheses; unaided-awareness survey design; the pipeline-funnel framing; success metric tied to value (human effort replaced), not vanity; clean bet, scoped MVP, explicit won't-build.

To tighten: stratify interviews instead of random sampling; lead with the headline ("distribution problem, not a model problem") before the evidence; state the workflow mechanism concretely (auto-post as PR comment on open) rather than "changing workflow"; name why stable repos (low variance, clean read); use behavioral trust proxies (edit distance, dismiss rate) instead of conceding trust to qual-only.

Executive-presence notes

- Headline first: "not a model problem - a distribution problem." Then the evidence.
- The cohort question is your differentiator. Ask it early and out loud.
- "What would you deliberately not build" is a test of whether you can say no with data. The funnel is your shield - point at the cliff.